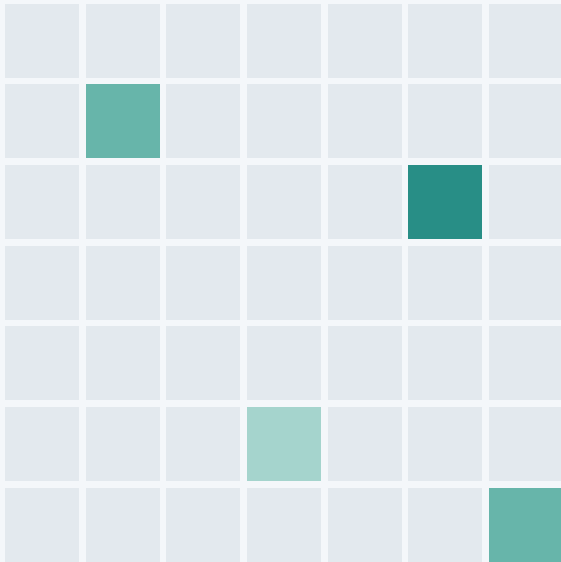


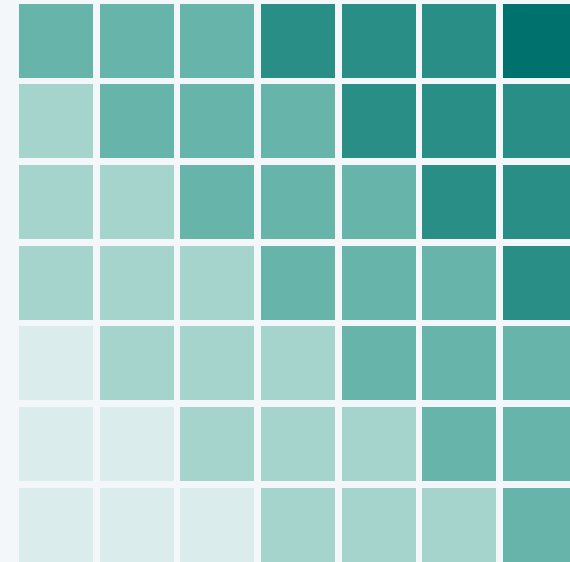
How can we predict light detection across the tank?

What we have



A few simulated locations

What we want



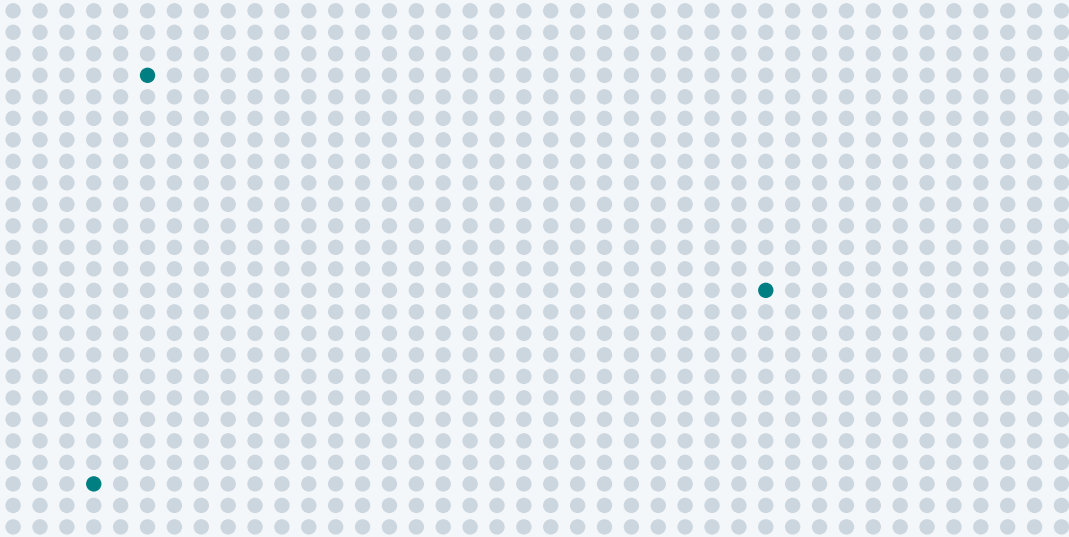
A probability at every location



Each square represents a voxel: one small region of the liquid argon. Schematic illustration.

One simulation tells us a count and a budget

One voxel • 1,000 simulated events



● Event with a detection

3

events produced a hit

$$3 / 1,000 = 0.3\%$$

The map should estimate the underlying chance of detection—not reproduce every random count.

Repeat the same simulation: the count changes

Same voxel. Same physics. Same 1,000 events per run.

0 hits

0.0%

2 hits

0.2%

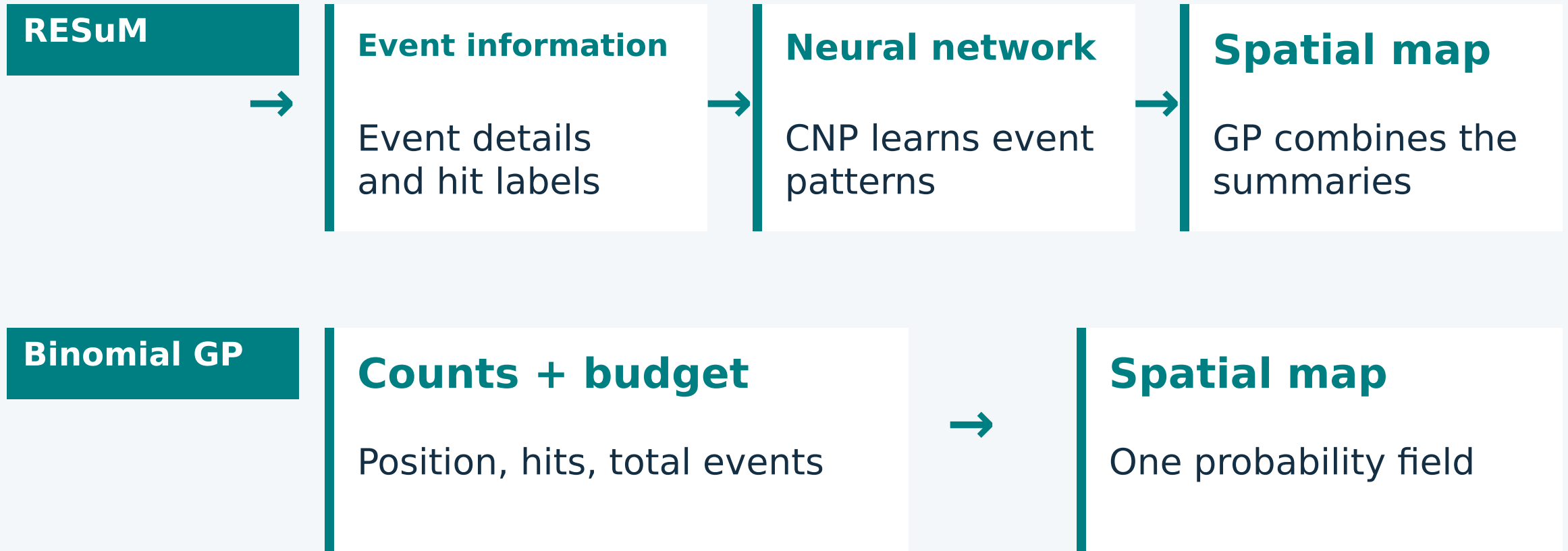
5 hits

0.5%

Different observations can come from the same probability.

This random variation is counting noise. Example values are illustrative.

Two routes from simulation data to an optical map



A GP is a model that learns a smooth spatial pattern and uncertainty from observed locations.

The binomial GP asks: what probability explains the counts?

Nearby voxel A

1 hit / 500 events



Nearby voxel B

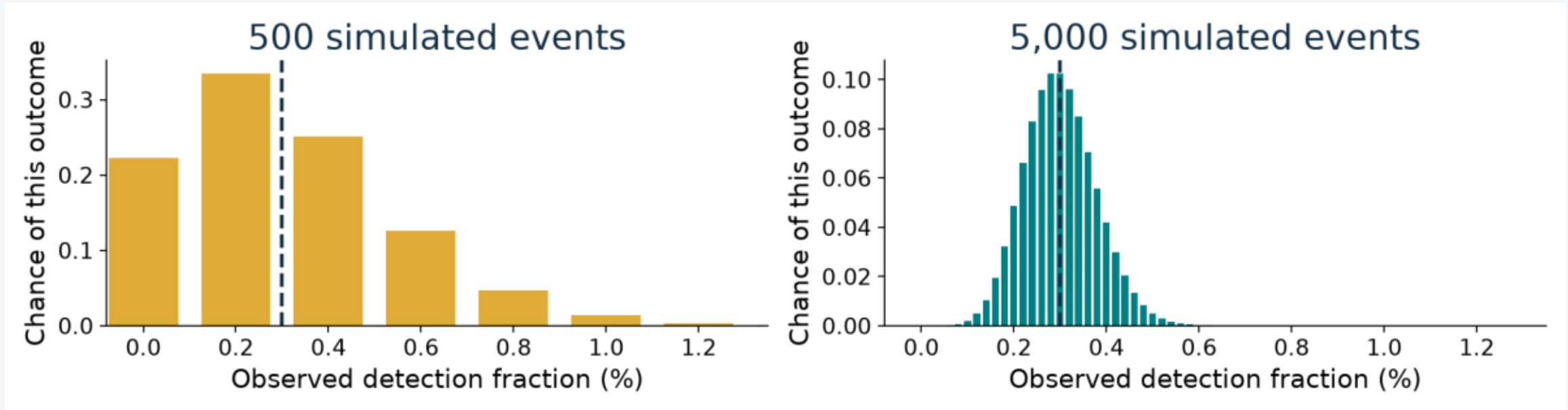
15 hits / 5,000 events

Learn the spatial pattern

Use each count together with its simulation budget.

More simulated events generally provide a more precise observation. No extra fidelity function is needed here.

A larger simulation gives a steadier estimate



Both simulations describe the same physics.

Illustration: the true detection probability is 0.3% in both panels. Dashed line = true probability.

Zero-hit simulations are useful information

0 hits

1,000 events

0 hits

1,000 events

1 hit

1,000 events

1 hit

1,000 events

3 hits

1,000 events

Keep all five runs

5 hits / 5,000 events = 0.10%

Keep only nonzero runs

5 hits / 3,000 events = 0.17%

Removing zero-hit files raises the observed average. A zero count does not mean zero probability.

Uncertainty answers two different questions

How well do we know the map?

Only a few locations were simulated.
The true probability is uncertain.



Uncertainty in the probability

What would a new run observe?

Even at a known probability,
a finite simulation produces random
counts.



Uncertainty in a new observed fraction

Both approaches must distinguish uncertainty in the map from variation between repeated simulations.

RESuM uses more information, but has more stages

Why keep RESuM?

Event features may explain detections.

The CNP can learn useful patterns before spatial interpolation.

It also provides event-level scores.

What must we check?

Denoising can introduce bias.

Mixup and training choices affect the averaged predictions.

Extra stages need extra validation.

The choice depends on what information we need

Binomial GP

Best starting point when:
we need a voxel probability map
and only simulation budgets differ.

Main strength: counts and N enter
directly.

Main limitation: no event-level
features.

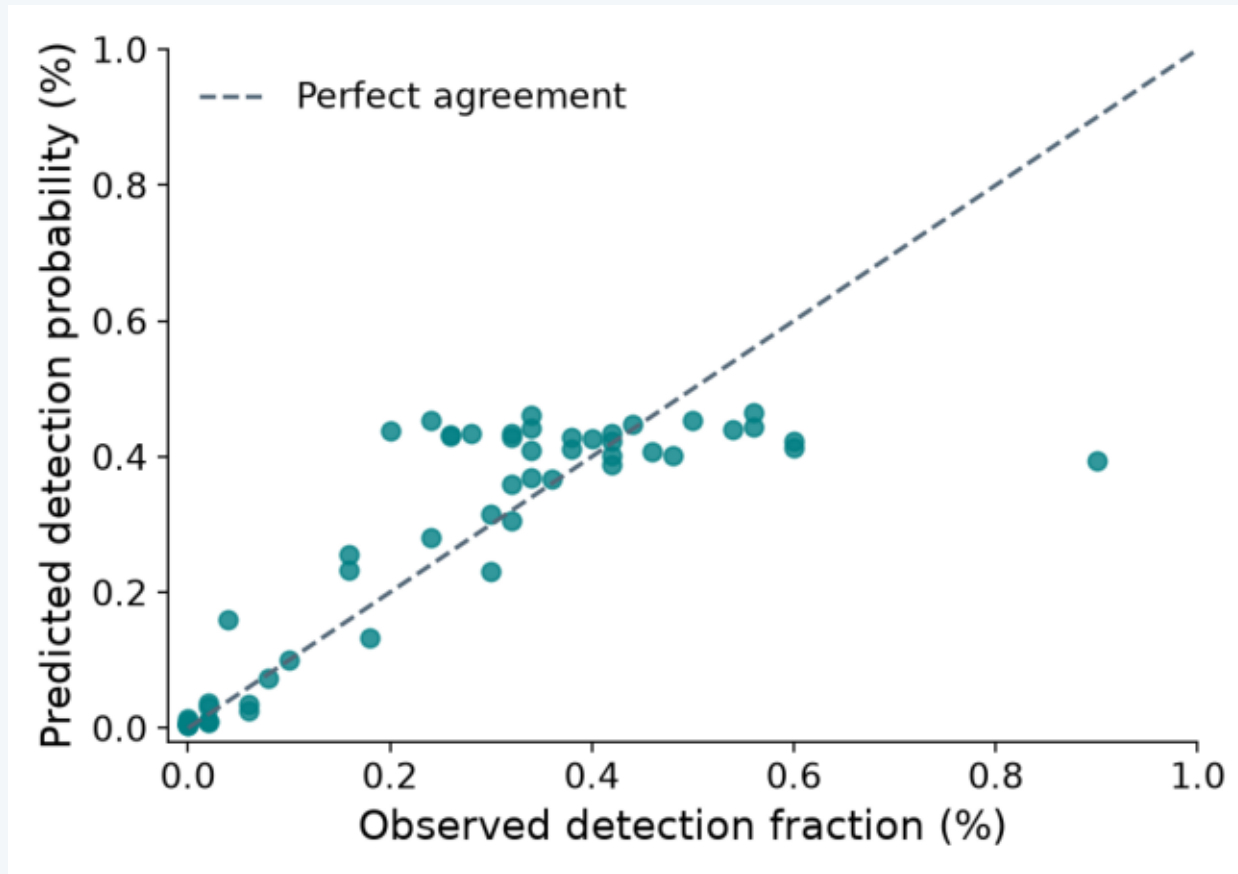
RESuM

Worth retaining when:
event features and CNP denoising
add measurable predictive value.

Main strength: richer input
information.

Main limitation: a more complex
pipeline.

The current model reproduces much of the spatial variation



55 held-out HF voxels

Each dot is one voxel.

Typical absolute error:
0.068 percentage points.

51 / 55 observations fall
inside
the nominal 95.45%
intervals.

This checks the binomial GP on its own data split. It does not establish superiority over RESuM.

The next test should be a fair comparison

- 1 Same data**
Same training files and held-out voxel locations.
- 2 Same target**
Same primary events, including zero-hit runs.
- 3 Three questions**
Are means accurate, intervals useful, and nearby voxels predicted well?

Use the binomial GP as the simple baseline; retain RESuM if its extra information improves matched tests.

Binomial GP: the statistical model

Prior: how probabilities vary in space

$$f(s) \sim \text{GP}(0, k)$$

$$p(s) = \text{sigmoid}[f(s)]$$

The kernel k links nearby locations.
The sigmoid keeps $0 < p < 1$.

Likelihood: how counts are produced

$$m_i \mid p(s_i), N_i \sim \text{Binomial}(N_i, p(s_i))$$

Each file contributes its count and budget.

Conditional counting variance:
 $\text{Var}(m_i / N_i \mid p) = p(1 - p) / N_i$

Posterior = prior spatial model updated by all observed counts.

Implemented inference: Laplace approximation; kernel parameters fitted on training data only.

RESuM: the statistical model

1 CNP: learn event probabilities

q_j = CNP-predicted hit probability

b_i = average of q_j within voxel i

Current training uses Bernoulli cross-entropy, with mixup.

2 MFGP: link the spatial summaries

$$g_l(s) = \rho_l g_{l-1}(s) + \delta_l(s)$$

$$z_l(s) = g_l(s) + \varepsilon_l, \varepsilon_l \sim \text{Normal}(0, \sigma_l^2)$$

ρ links levels; δ is a GP correction.
 σ_l^2 is learned noise for each level.

Three log-space outputs: LF CNP mean • HF CNP mean • HF raw rate*

* HF training rates are smoothed before taking logs. The MFGP likelihood is Gaussian, not binomial.

Technical appendix: probability and counting uncertainty

1 Spatial pattern

$$f(s) \sim \text{GP}(0, k)$$

$$p(s) = 1 / [1 + \exp(-f(s))]$$

2 Counts at each voxel

$$m_i \sim \text{Binomial}(N_i, p(s_i))$$

N_i is known; $p(s_i)$ is inferred.

Predicted fraction variance at an exact location:

$$\text{Var}(R \mid s, D) = \text{Var}(P \mid s, D) + E[P(1 - P) \mid s, D] / N$$

First term: uncertainty in the map. Second term: counting noise. Inference uses a Laplace approximation.

Technical appendix: the saved runs differ

	Saved RESuM	Saved binomial GP
LF training	159 nonzero N1500 voxels	760 across N500–N1500; includes zeros
HF training	10 nonzero voxels	10 voxels, including 1 zero-hit
HF validation	46 nonzero voxels	55 voxels, including 8 zero-hit
HF primary budget	4936 after context removal	Full 5000

Different data and targets can change results independently of the modeling approach.

References and provenance

Background

Conditional Neural Processes — Garnelo et al. (2018)
<https://proceedings.mlr.press/v80/garnelo18a.html>

Gaussian Processes for Machine Learning — Rasmussen & Williams, Ch. 3
<https://gaussianprocess.org/gpml/chapters/RW3.pdf>

Local implementation and results

RESUM_FLEX: core/binomial_gp.py and core/surrogates/mfgp_stage.py
Count-GP run 20261001T173754_521023Z • RESuM run 20260930T195141_282030Z